pynasonde — Methods & Capabilities Overview

An open-source Python library for ionosonde data processing

Chakraborty et al., *SoftwareX* 34, 102617 (2026)

S. Chakraborty T. Bullett A. Barjatya J. Mabie

Embry-Riddle Aeronautical University — Fourth State Communications

July 21, 2026

`pip install pynasonde` — github.com/shibaji7/pynasonde — pynasonde.readthedocs.io

Outline

1. What is pynasonde?
2. Architecture
3. DIGISONDE Module
4. VIPIR Module
5. Analysis Layer
6. Es Layer Imaging (In Development)
7. New Capabilities vs. Roadmap
8. Complete API Reference

Why pynasonde?

- **Problem**: Digisonde tools (Java) and VIPIR tools are instrument-specific, fragmented, and not always publicly available
- **Solution**: A unified, open-source Python framework for *both* instrument families
- **Key design goals**:
  - Modular architecture (read → plot → analyse)
  - Support 9 file formats (binary + ASCII + XML)
  - Lower barrier to reproducible analysis
  - Built-in analysis pipeline, not just plotting
- **Published**: Chakraborty et al., *SoftwareX* 34, 102617 (2026)
- **Current**: v1.0 — substantially extended beyond publication

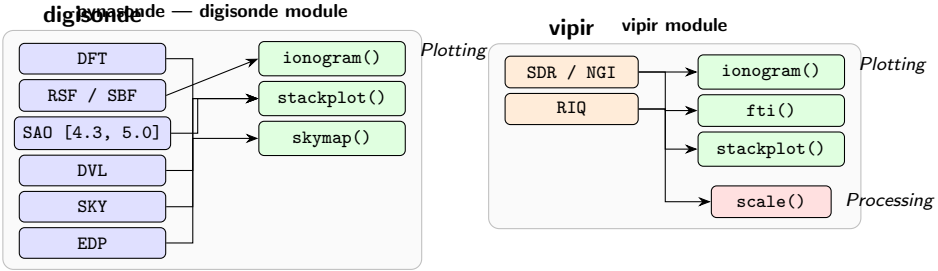
Installed capabilities

12 file formats read (incl. CADI, GRM)
10 digisonde parsers + cross-format scaler
8 VIPIR I/O classes (echo, filter, scale, polarize)
7 analysis modules *(in development)*
25+ public API classes
Full pipeline automation

Cite as

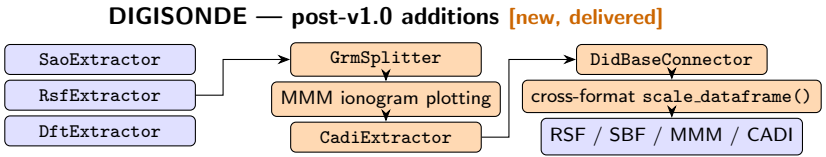
Chakraborty et al. (2026).
SoftwareX 34, 102617.
DOI: 10.1016/j.softx.2026.102617
Zenodo: 10.5281/zenodo.17476077

pynasonde v1.0 Architecture (Chakraborty et al. 2026)



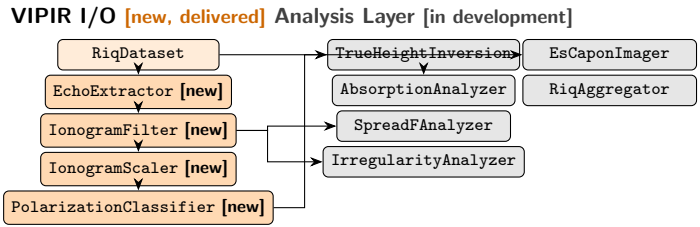
Adapted from Fig. 2 of Chakraborty et al. (2026), *SoftwareX* 34, 102617.

Extended Architecture — Digisonde I/O & Scaling [delivered]



GRM auto-splits into RSF/SBF/MMM; MMM plotting, CADI echo extraction, DIDBase access, and cross-format scaling are new since v1.0 and are delivered, tested features of this release.

Extended Architecture — VIPIR I/O [delivered] & Analysis Layer [in development]



Orange = new, delivered VIPIR I/O (echo extraction, filtering, scaling, polarization). Gray = analysis-layer modules present in the codebase but **not yet validated/released** — shown as a preview of ongoing work, not a claim of a stable API.

DIGISONDE Module — File Formats & Parsers

Ext.	Type	Parser class	Key methods / output
.SAO 4.3	ASCII	SaoExtractor	load_SAO_files(): height_profile / scaled_params
.SAO 5.0	XML	SaoExtractor	same API; SaoSummaryPlots.plot_TS()
.RSF	Binary	RsfExtractor	plot_direction_ionogram(), plot_directogram()
.SBF	Binary	SbfExtractor	extract(), ionogram()
.DFT	ASCII	DftExtractor	plot_doppler_waterfall(), plot_doppler_spectra()
.DVL	ASCII	DvlExtractor	load_DVL_files(), plot_dvl_drift_velocities()
.SKY	ASCII	SkyExtractor	extract(), to_pandas(), plot_skymap()
.EDP	ASCII	EdpExtractor	extract(), electron density profile
.GRM	Binary	GrmSplitter **[new]**	auto-detects & splits RSF/SBF/MMM archives from GIRO DIDBase
.MMM	Binary	ModMaxExtractor	O/X pcolormesh ionogram plotting **[new]**
MD2/MD4	Binary	CadiExtractor **[new]**	product= "products"|"echoes": amplitude, phase, $\Delta\phi_{ij}$, coherence

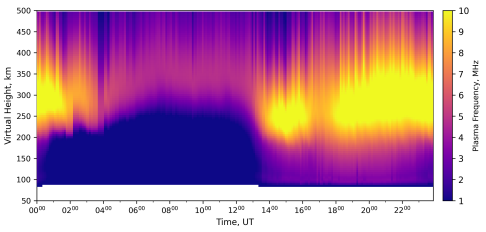
DIGISONDE — SAO: Density Profiles & Scaled Parameters

```python
from pynasonde.digisonde.parsers.sao \
    import SaoExtractor, SaoSummaryPlots

# N(h) profiles for a whole day
df = SaoExtractor.load_SAO_files(
    folders=["data/KR835/"],
    func_name="height_profile",
    n_procs=4,
)
# cols: datetime, height_km,
#       plasma_freq_mhz, Ne_cm3

# Scaled ionospheric parameters
params = SaoExtractor.load_SAO_files(
    folders=["data/KR835/"],
    func_name="scaled_params",
)
# cols: datetime, foF2, hmF2, foE, MUF

SaoSummaryPlots(df).plot_TS(
    params=["foF2","hmF2"])
```



Daily isodensity contour, KR835.

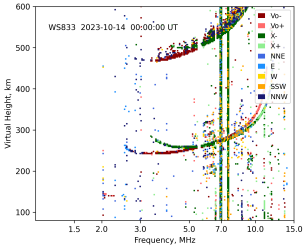
DIGISONDE — RSF: Direction-Coded Ionogram

```python
from pynasonde.digisonde.parsers.
    rsf \
    import RsfExtractor

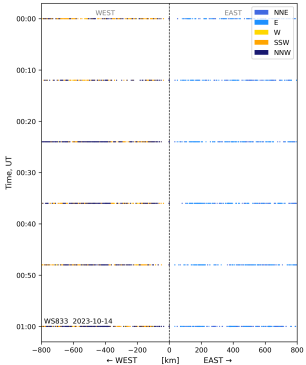
ex = RsfExtractor("KR835.RSF")
ex.extract()

# colour-coded by arrival azimuth
ex.plot_direction_ionogram()

# full day: TID / ground distance
ex.plot_directogram()
```



Direction ionogram



Directogram (daily)

What RSF adds

- Arrival azimuth per echo (4 Rx)
- Full amplitude + phase
- Directogram = TID signature

DIGISONDE — DFT, DVL, SKY

DFT

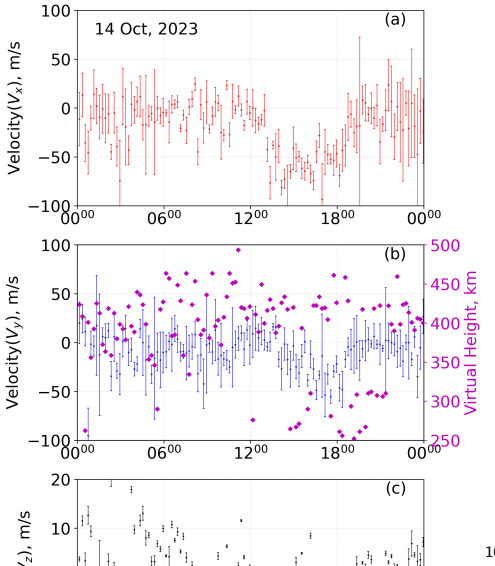
```
dft = DftExtractor("KR835.DFT")
dft.extract()
dft.plot_doppler_waterfall()
```

DVL

```
dvl = DvlExtractor.load_DVL_files(
    folder="data/KR835/", n_procs=4)
dvl.plot_dvl_drift_velocities()
# panels: Vx(E), Vy(N), Vz(up)
```

SKY

```
sky = SkyExtractor("KR835.SKY")
sky.extract()
SkySummaryPlots(sky.to_pandas())\
    .plot_skymap()
```



DIGISONDE — GRM/RSF/MMM Plotting & CADI Echo Extraction

GRM auto-split

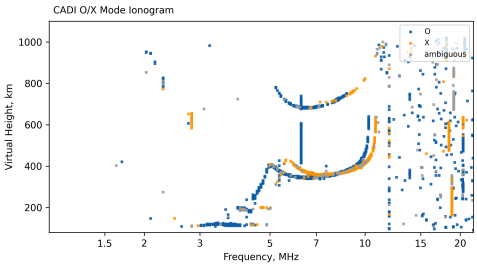
```
from pynasonde.digisonde.parsers.grm \
    import GrmSplitter

spl = GrmSplitter("AU930.GRM")
print(spl.fmt)   # "RSF"|"SBF"|"MMM"
df = spl.load_dataframes()
```

CADI interferometric echoes

```
from pynasonde.digisonde.cadi \
    import CadiExtractor

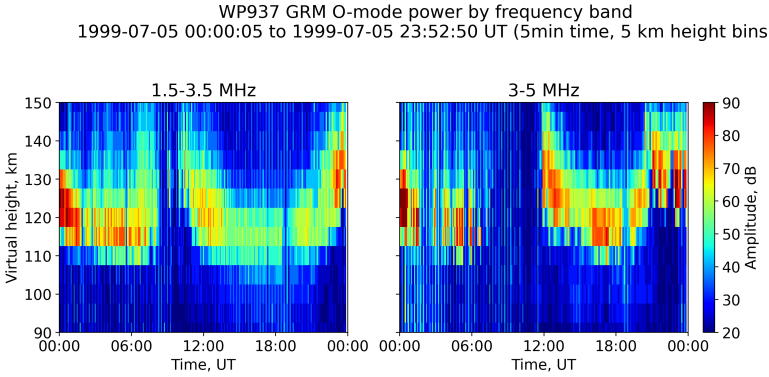
df = CadiExtractor.load_CADI_files(
    folders=["tmp/CADI/"],
    product="echoes", n_procs=1)
# cols: dphi_12, dphi_13, dphi_23,
#       coherence, mode
```



CADI O/X mode separation from cross-receiver baseline phase. MMM ionograms render as O/X pcolormesh (see `docs/examples/mmm.md`).

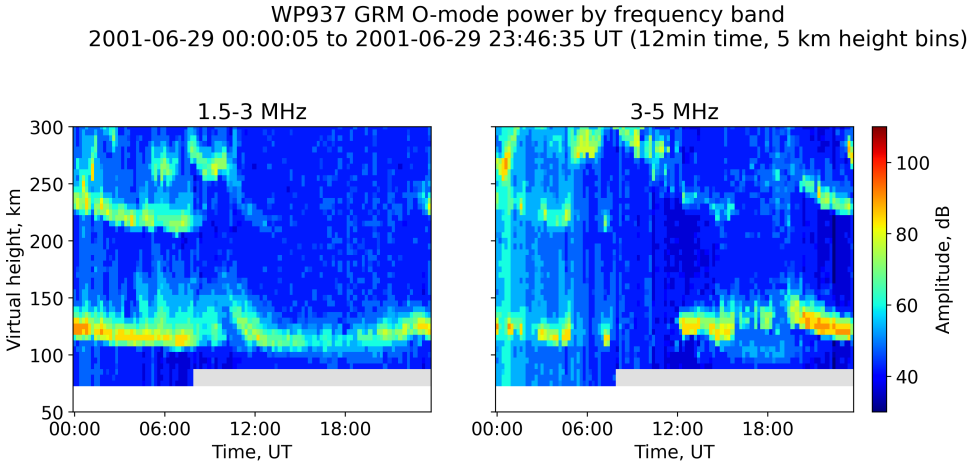
Special Analysis — Long-Term O-Mode Power Directogram (RSF/MMM)

GRM archives let decades-old WP937 soundings (RSF- and MMM-era DPS hardware alike) be binned onto a common, d256-compatible virtual-height grid (5 km bins — the classic 256-bin DPS height-gate table) and a regular time grid, then split by frequency band to reveal diurnal E/Es persistence across a full day:



WP937, 1999-07-05: O-mode power (dB) vs. virtual height and UT, 1.5–3.5 and 3–5 MHz bands, 5-min × 5-km bins.

Special Analysis — Cross-Era Comparison (1999 vs. 2001)



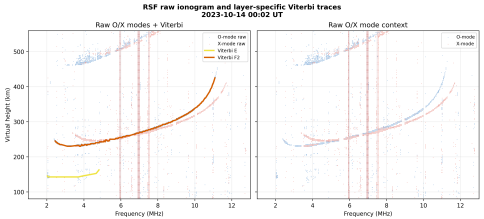
WP937, 2001-06-29: same binning scheme, 12-min cadence (lower duty cycle that day). The persistent 110–140 km E/Es layer and its diurnal descent/ascent are directly comparable to the 1999 case despite the different source hardware and time resolution — the point of a d256-compatible grid. Gray band = no receiver coverage below ∼90 km that day.

DIGISONDE — DIDBase Access & Cross-Format Scaling

```python
from pynasonde.digisonde.didbase \
    import DidBaseConnector
from pynasonde.digisonde.scaler \
    import scale_dataframe

with DidBaseConnector(
        url, user, pwd, role) as db:
    raw = db.fetch_dataframe(
        station="AU930",
        start="2017-05-27",
        end="2017-05-27", fmt="MMM")

# same call for RSF, SBF, MMM, CADI
scaled, _ = scale_dataframe(
    raw, mode="O", method="viterbi")
```



One schema-detecting autoscaler (RANSAC/DBSCAN-regularised Viterbi trace tracking) covers the full RSF/SBF/MMM/CADI echo-table family.

VIPIR Module — File Formats & I/O Classes

Ext.	Type	I/O class	Key methods
.RIQ	Binary	`RiqDataset`	`create_from_file()`, SCT + PCT, `VIPIR_VERSION_MAP`
.RIQ	—	`EchoExtractor` **[new]**	7-param echoes (h, A, V*, EP, PP, XL, YL); `to_dataframe()`
.RIQ	—	`IonogramFilter` **[new]**	6-stage: RFI, EP, multi-hop, DBSCAN, RANSAC, temporal
.NGI	NetCDF	`DataSource`	`load()`, multi-SDR data cube aggregation
.NGI	—	`AutoScaler`	noise profile, segmentation, binary trace extraction
.RIQ	—	`IonogramScaler` **[new]**	automated URSI/CCIR: foE, foF2, h'F, MUF(3000), M(3000)F2
.RIQ	—	`PolarizationClassifier` **[new]**	O/X wave-mode labelling via PP chirality, hemisphere-aware
.RIQ	—	`FullPipeline` **[new]**	end-to-end: RIQ → echoes → filter → O/X → N(h)

Supported stations

WI937 — Wallops Island (data_type=1)

New since v1.0

EchoExtractor + IonogramFilter + IonogramScaler + PolarizationClassifier + FullPipeline — the complete

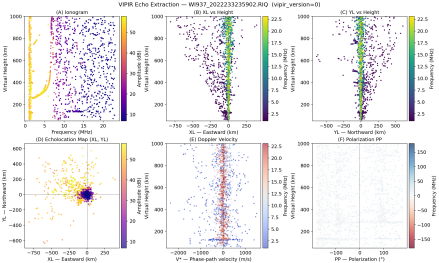
VIPIR — RIQ Loading, Echo Extraction & Filtering

```
from pynasonde.vipir.riq.parsers.read_riq \
    import RiqDataset, VIPIR_VERSION_MAP
from pynasonde.vipir.riq.echo \
    import EchoExtractor
from pynasonde.vipir.riq.parsers.filter \
    import IonogramFilter

# 1. Load raw IQ
riq = RiqDataset.create_from_file(
    "WI937_20220821.RIQ",
    vipir_config=VIPIR_VERSION_MAP.configs
        [1])
# gates=960, r0=1.499 km, 8Rx, 4 pulses/freq

# 2. Extract 7-parameter echoes
ex = EchoExtractor(
    sct=riq.sct, pulsets=riq.pulsets,
    snr_threshold_db=3.0).extract()
df = ex.to_dataframe()
# cols: freq_mhz, height_km, amplitude_db,
#       doppler_mps, EP_deg, PP_deg, XL, YL

# 3. Six-stage noise filter
```



6-panel: ionogram (A), XL (B), YL (C), echo map (D), Doppler V* (E), PP (F).

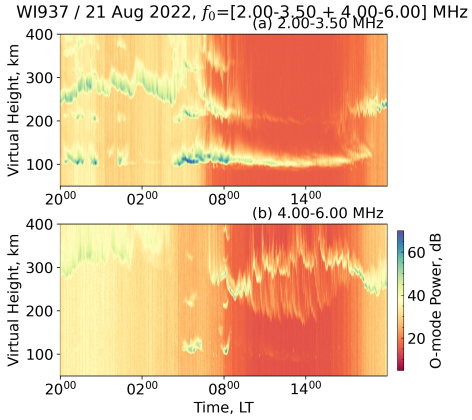
VIPIR — NGI/SDR: FTI & AutoScaler

```
from pynasonde.vipir.ngi.source \
    import DataSource
from pynasonde.vipir.ngi.scale \
    import AutoScaler

# Load NGI data cube
ds = DataSource(files=sdr_file_list)
ds.load()

# Frequency-binned time-intensity plot
ds.fti(freq_bands=[
    (2.0, 3.5),
    (4.0, 6.0)])

# Automated scaling
scaler = AutoScaler(ds)
scaler.run()
# noise profile -> segment -> binary
# -> DBSCAN trace extraction
params = scaler.scaled_parameters
# .foF2, .foE, .hpF, .MUF, ...
```



FTI from WI937: persistent Es at 100 km; intensifies 08–12 UT.

Analysis Layer — Seven Modules (In Development)

Implemented in the codebase but not yet validated/released — shown as a preview of ongoing work. `PolarizationClassifier` and `IonogramScaler` have already graduated to the delivered VIPIR module (previous section).

Class	Module	What it does
`TrueHeightInversion`	`inversion`	Virtual → true height via Titheridge lamination (POLAN)
`AbsorptionAnalyzer`	`absorption`	LOF, differential O/X, calibrated total, height-resolved $\kappa(z)$
`SpreadFAnalyzer`	`spread_f`	Range / frequency / mixed spread-F detection & classification
`IrregularityAnalyzer`	`irregularities`	EP structure function → spectral index $\alpha(h)$, outer scale
`EsCaponImager`	`es_imaging`	Capon cross-spectrum: 10× range resolution, single file
`RiqAggregator`	`es_imaging`	Multi-file A+B+C combining: ∼24 dB SNR, slow RTI

Import: `from pynasonde.vipir.analysis import` *`ClassName`*

O/X Polarization Separation [Delivered — VIPIR Module]

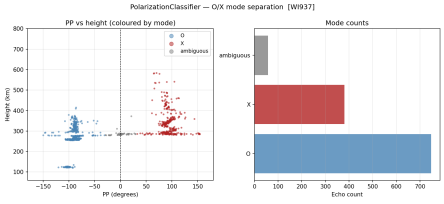
```python
from pynasonde.vipir.analysis import (
    PolarizationClassifier,
)

# O/X mode separation via PP chirality
pol = PolarizationClassifier(
    o_mode_sign=-1,   # N. hemisphere
    pp_ambiguous_threshold_deg=20,
).fit(filtered_df)

print(pol.summary())
# O=412  X=388  ambiguous=54

# Get O-mode only DataFrame
o_df = pol.o_mode_df()

# pol.annotated_df has 'mode' column
# (values: 'O', 'X', 'ambiguous')
```



PP histogram with O/X classification regions.

Key parameters

`o_mode_sign`: -1 (N. hemisphere) / $+1$ (S. hemisphere)

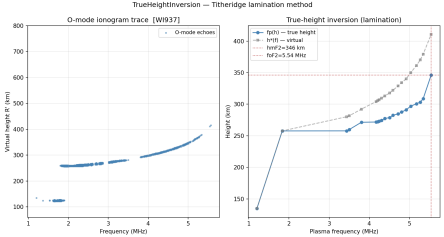
True Height Inversion — N(h) Profile [In Development]

```python
from pynasonde.vipir.analysis import (
    TrueHeightInversion,
)

# Titheridge lamination (POLAN)
inv = TrueHeightInversion(
    monotone_enforce=True,
    bin_width_mhz=0.05,
)
# O-mode filter applied automatically
edp = inv.fit_from_df(o_df)

print(edp.summary())
# foF2=8.12 MHz
# hmF2=287.4 km
# NmF2=8.2e5 cm^-3

edp.plot()
```



N(h) profile and virtual vs. true height comparison.

Key parameters

`bin_width_mhz`: frequency decimation for numerical stability

HF Absorption — Four Estimators [In Development]

```python
from pynasonde.vipir.analysis import (
    AbsorptionAnalyzer,
)
import numpy as np

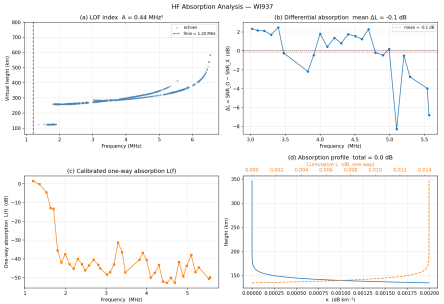
aa = AbsorptionAnalyzer(f_ref_mhz=1.0)

# 1. LOF index (no calibration needed)
lof  = aa.lof_absorption(filtered_df)

# 2. Differential O/X SNR
diff = aa.differential_absorption(
           pol.annotated_df)

# 3. Calibrated total absorption (dB)
total= aa.total_absorption(
           filtered_df, tx_eirp_dbw=70.0)

# 4. Height-resolved kappa(z) profile
prof = aa.absorption_profile(
    edp,
    nu_hz=lambda h: 1e6*np.exp(-h/8.0))
```



(a) LOF, (b) O/X differential, (c) calibrated total, (d) $\kappa(z)$ profile.

Spread-F & Irregularity Analysis [In Development]

Spread-F — type classification

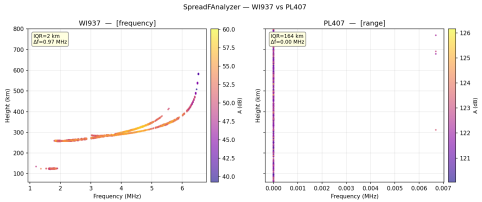
```
from pynasonde.vipir.analysis import (
    SpreadFAnalyzer,
    IrregularityAnalyzer,
)

sf = SpreadFAnalyzer().fit(filtered_df)
print(sf.summary())
# type=range  foEs=4.2 MHz
# h_spread_km=38.1
```

Irregularities — spectral index profile

```
irr = IrregularityAnalyzer(
    height_bin_km=10.0,
    min_echoes_per_bin=5,
).fit(filtered_df)

# result: alpha(h), outer_scale_km,
#         anisotropy proxy 0/X
irr.plot_spectral_profile()
```



Spread-F detection: type + foEs + height spread metrics.

Sporadic-E Imaging — Capon Cross-Spectrum (Liu et al. 2023) [In Development]

This entire section is implemented in the codebase but not yet a validated/released feature.

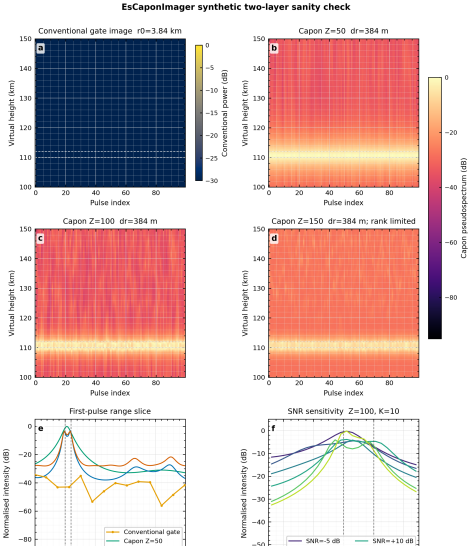
Physical motivation

- Es: thin metallic-ion layer, 90–130 km, <1 km thick
- VIPIR gate: $r_0 = 1.499$ km — cannot resolve
- Capon (Liu et al. 2023): **10× finer resolution**, no extra hardware required
- Effective resolution: $\Delta r = r_0/K$

Resolution

	r_0	Δr (K=10)
WISS	3.84 km	384 m
VIPIR	1.499 km	**150 m**

Key constraint (Liu et al.)



Es Imaging — Single-File Sanity Check (Real Data)

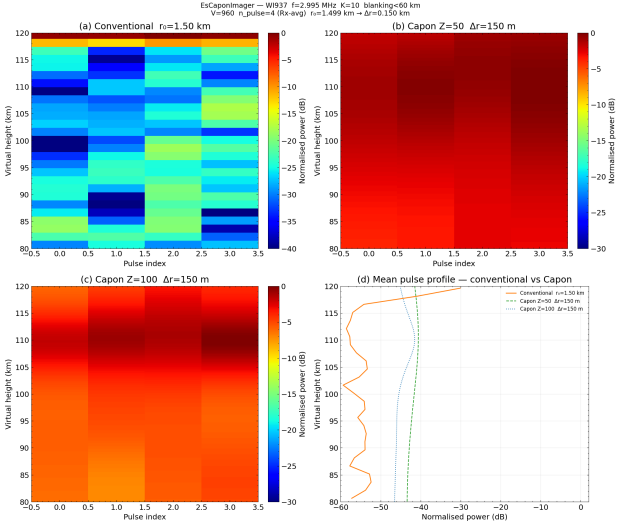
```
# 1. Load closest pulset, avg 8 Rx
cube = _load_cube(
    "WI937.RIQ",
    freq_khz=3000.0)     # (4, 960)

# 2. Blank below 60 km (direct
    wave)
blank = int((60 - gate_start) / r0
    )
cube[:, :blank] = 0

# 3. Capon Z=50 and Z=100
res50  = EsCaponImager(
    n_subbands=50,  K=10).fit(cube
        )
res100 = EsCaponImager(
    n_subbands=100, K=10).fit(cube
        )
```

Pipeline

RIQ → Rx-avg → blank < 60 km
→ Capon (Z=50, Z=100, K=10)



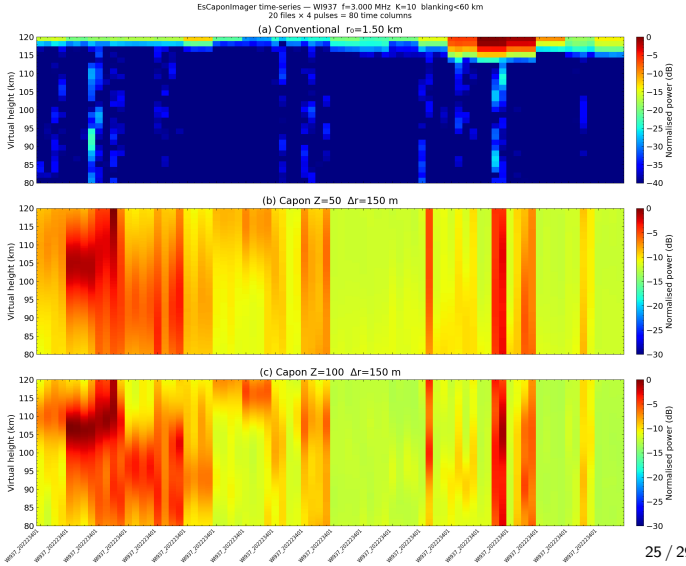
(a) Conventional, (b) Capon Z=50, (c) Capon Z=100, (d) mean

Es Imaging — 20-File Time-Series RTI

```
from concurrent.futures import
    (
    ThreadPoolExecutor,
    as_completed)

ordered = {}
with ThreadPoolExecutor() as
    ex:
    futures = {
        ex.submit(
            _process_file, src
                ): i
        for i, src in
            enumerate(
            paths[:20])}
    for f in as_completed(
        futures):
        i = futures[f]
        ordered[i] = f.result
            ()

# re-sort, concatenate -> (80,
    n_hr)
```



What Has Been Added Since v1.0 Publication

Digisonde I/O & scaling (delivered)

- `GrmSplitter` — GRM → RSF/SBF/MMM auto-split
- MMM ionogram plotting (O/X pcolormesh)
- `CadiExtractor` — CADI MD2/MD4, interferometric echoes
- `DidBaseConnector` — GIRO DIDBase remote access
- `scale_dataframe()` — one autoscaler across RSF/SBF/MMM/CADI

VIPIR signal-processing chain (delivered)

- `EchoExtractor` — 7-param Dynasonde echoes
- `IonogramFilter` — 6-stage pipeline
- `IonogramScaler` — foE, foF2, h'F, MUF(3000)
- `PolarizationClassifier` — O/X separation
- `FullPipeline` — end-to-end automation

Analysis layer (in development)

- `TrueHeightInversion`, `AbsorptionAnalyzer`
- `SpreadFAnalyzer`, `IrregularityAnalyzer`
- `EsCaponImager`, `RiqAggregator`

Versioning roadmap status

Ver.	Planned item	Status
v2.0	CADI support	Done
v2.0	GRM/MMM plotting	Done
v2.0	DIDBase access	Done
v2.0	Cross-format scaler	Done
v2.0	Oblique ionograms	Partial
v3.0	Directional info	Done
v3.0	Polarization info	Done
v4.0	Cisium plotting	Planned
Extra	Modified POLAN	In dev.
Extra	Es Capon imaging	In dev.
Extra	Absorption analysis	In dev.
Extra	Irregularities	In dev.

v2.0 is ready to release on the strength of the delivered Digisonde and VIPIR items above. The analysis layer continues toward validation as a subsequent release.

Complete Public API — All Classes

DIGISONDE I/O & Scaling

`SaoExtractor`	SAO profiles + params
`SaoSummaryPlots`	Isodensity, time-series
`RsfExtractor`	Direction ionogram, directogram
`SbfExtractor`	SBF binary
`DftExtractor`	Doppler waterfall + spectra
`DvlExtractor`	Drift velocities
`SkyExtractor`	SKY polar maps
`SkySummaryPlots`	Multi-panel skymap
`EdpExtractor`	Electron density profile
`GrmSplitter`	**[new]** RSF/SBF/MMM auto-split
`ModMaxExtractor`	MMM O/X ionogram
`CadiExtractor`	**[new]** CADI MD2/MD4, echoes
`DidBaseConnector`	**[new]** GIRO remote access
`scale_dataframe()`	**[new]** RSF/SBF/MMM/CADI scaling

VIPIR I/O

`RiqDataset`	Load .RIQ + SCT/PCT
`EchoExtractor`	**[new]** 7-param echoes
`IonogramFilter`	**[new]** 6-stage filter
`IonogramScaler`	**[new]** foE, foF2, h'F, MUF
`PolarizationClassifier`	**[new]** O/X via PP
`DataSource`	NGI/SDR data cube
`AutoScaler`	Segment + scale

Analysis Layer (in development)

`TrueHeightInversion`	N(h), foF2, hmF2
`EDPResult`	profile dataclass
`AbsorptionAnalyzer`	4 estimators
`LOFResult`	LOF index
`SpreadFAnalyzer`	Type + metrics
`SpreadFResult`	classification
`IrregularityAnalyzer`	$\alpha(h)$, L_{outer}
`EsCaponImager`	Capon 1-file
`EsImagingResult`	pseudospectrum
`RiqAggregator`	Multi-file RTI

Implemented, not yet validated/released.

pynasonde.readthedocs.io

Summary

pynasonde v2 delivers an end-to-end Digisonde + VIPIR workflow:

1. Read **12 file formats** across three instrument families (DIGISONDE, VIPIR, CADI)
2. **GRM/RSF/MMM ionogram plotting**, corrected **DFT** reader, **DIDBase** remote access
3. **CADI support** with interferometric echo extraction
4. One **cross-format autoscaler** spanning RSF/SBF/MMM/CADI
5. **VIPIR**: 7-parameter echo extraction, automated URSI/CCIR scaling, O/X polarization classification
6. **In development**: true-height inversion, spread-F/irregularity characterisation, Capon Es imaging

Quick start

```
pip install pynasonde
```

github.com/shibaji7/pynasonde

pynasonde.readthedocs.io

Cite as

Chakraborty et al. (2026).
SoftwareX 34, 102617.
DOI: 10.1016/j.softx.2026.102617

Next

Validate and release the analysis layer (true-height inversion, Es Capon imaging)

References I

- Chakraborty S., Bullett T., Barjatya A., Mabie J. (2026). pynasonde: An open-source Python library for ionosonde data processing. *SoftwareX* 34, 102617.
- Liu, T., Yang, G., & Jiang, C. (2023). High-resolution sporadic E via Capon cross-spectrum. *Space Weather* 21, e2022SW003195.
- Capon, J. (1969). High-resolution frequency-wavenumber spectrum analysis. *Proceedings of the IEEE* 57(8), 1408–1418.
- Titheridge, J. E. (1967). *JATP* 29, 763–778.
- Reinisch, B. W., & Galkin, I. A. (2011). Global Ionospheric Radio Observatory (GIRO). *Earth, Planets and Space* 63, 377–381.
- Grubb, R. N. et al. (2008). VIPIR. *URSI General Assembly*.
- Reinisch, B. W. et al. (2009). New Digisonde. *Radio Science* 44.
- Chakraborty S., Qian L., Mrak S. et al. (2025). G-condition 2017 GAE. *Earth and Space Science* 12, e2024EA004007.